

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Diseño Web II

Entrega 5A

API y Listado de Materias

FASE 2 — Gestión de Usuarios y Materias

Abril 2026

Proyecto: gestion-academica

Objetivos de Aprendizaje

Al completar esta entrega, el estudiante será capaz de:

- Crear esquemas de validación con Zod para entidades del dominio (materias)
- Implementar una API Route GET que realiza JOIN entre tablas relacionadas
- Implementar una API Route POST con validación Zod en el servidor
- Construir un componente Client que lista datos tabulares con filtros
- Crear un formulario de alta con Select que carga datos dinámicos (lista de docentes)
- Usar el componente Dialog de shadcn/ui para formularios modales
- Comprender el patrón de relaciones FK y cómo representarlas en la interfaz
- Aplicar toast de Sonner para feedback de operaciones CRUD

Prerrequisitos

Antes de comenzar, asegúrate de haber completado:

- ✓ Entregas 1A a 4A completadas y funcionando
- ✓ Entrega 4B — Panel Admin: Gestión de Usuarios
- ✓ Al menos 1 usuario con rol "admin" para acceder al panel
- ✓ Al menos 1 usuario con rol "docente" para asignar a materias
- ✓ Tabla "materias" creada en la base de datos (Entrega 1B)

i Recordatorio: La tabla materias tiene estos campos: id (UUID), codigo (TEXT UNIQUE), nombre (TEXT), credits (INTEGER 1-10), docente_id (UUID FK a usuarios_perfil), semestre (TEXT), created_at (TIMESTAMPZ). Revisa la Entrega 1B si necesitas confirmar el esquema.

Teoría: Relaciones FK y Validación en Servidor

Foreign Keys en la Interfaz

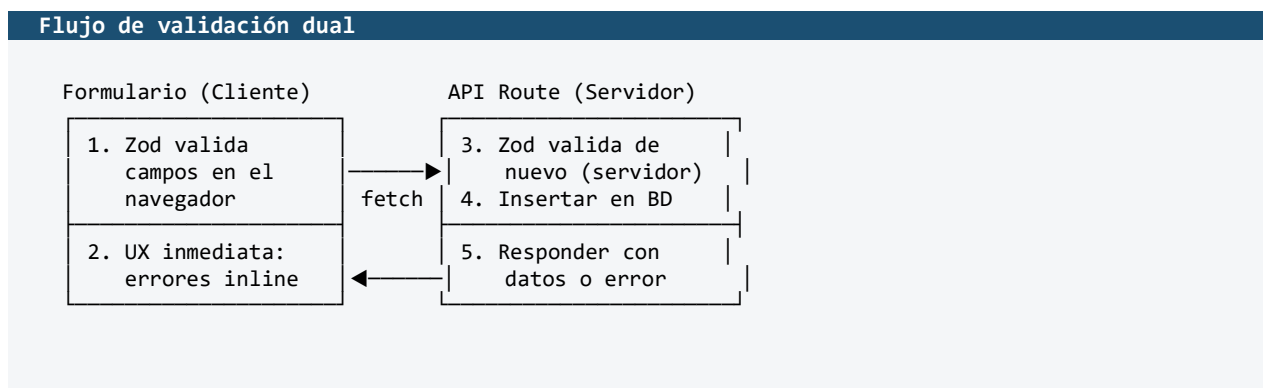
La tabla materias tiene una columna docente_id que es una Foreign Key (FK) hacia usuarios_perfil. Esto crea una relación uno-a-muchos: un docente puede tener muchas materias, pero cada materia tiene a lo sumo un docente.

En la interfaz, esta relación se traduce en:

En la BD	En la API	En la UI
docente_id (UUID)	JOIN con usuarios_perfil	Nombre del docente en la tabla
FK constraint	Validar que el ID existe	Select con lista de docentes
NULL permitido	docente_id es opcional	Opción "Sin asignar" en el Select

Validación Dual: Cliente y Servidor

En la Entrega 4A validamos el perfil solo en el cliente con Zod + React Hook Form. Ahora agregaremos validación también en el servidor. Esto es crucial para seguridad:



⚠ Importante: La validación en el cliente es para UX (feedback instantáneo). La validación en el servidor es para seguridad (un atacante puede saltarse el cliente). Siempre valida en ambos lados.

Supabase SELECT con relaciones

Supabase permite hacer JOINS implícitos usando la sintaxis de select con nombre de tabla relacionada:

Ejemplo: SELECT con relación FK

```
// Sin relación – solo obtienes el UUID del docente
const { data } = await supabase.from("materias").select("*");
// data[0].docente_id = "a1b2c3d4-..." ← UUID, no útil para mostrar

// Con relación – obtienes los datos del docente
const { data } = await supabase
  .from("materias")
  .select("*", docente:usuarios_perfil!docente_id(id, nombre, apellido));
// data[0].docente = { id: "a1b2c3", nombre: "Juan", apellido: "Pérez" }
```

☑ **Sintaxis:** El formato es: alias:tabla_relacionada!columna_fk(campos). El alias "docente" es como llamaremos al objeto en el resultado JSON.

Paso 1 — Instalar Componentes Nuevos

Necesitamos 2 componentes nuevos de shadcn/ui para esta entrega:

Terminal

```
npx shadcn@latest add dialog  
npx shadcn@latest add textarea
```

Componente	Archivo generado	Para qué lo usaremos
Dialog	components/ui/dialog.tsx	Formulario modal para crear materia
Textarea	components/ui/textarea.tsx	Campo de descripción (ejercicio propuesto)

i Nota: Los componentes Table, Badge, Select y AlertDialog ya los instalaste en la Entrega 4B. Los reutilizaremos aquí.

Paso 2 — Esquema de Validación Zod para Materias

Antes de crear las API Routes, definiremos el esquema de validación. Este esquema se usará tanto en el servidor (API Route) como en el cliente (formulario).

2.1 — Crear el archivo de validación

Crea el archivo `lib/validations/materia.ts`:

`lib/validations/materia.ts`

```
import { z } from "zod";

export const materiaSchema = z.object({
  codigo: z
    .string()
    .min(3, "El código debe tener al menos 3 caracteres")
    .max(10, "El código no puede exceder 10 caracteres")
    .regex(/^[A-Z0-9-]+$/, "Solo mayúsculas, números y guiones")
    .transform((val) => val.toUpperCase()),

  nombre: z
    .string()
    .min(3, "El nombre debe tener al menos 3 caracteres")
    .max(100, "El nombre no puede exceder 100 caracteres")
    .transform((val) => val.trim()),

  credits: z
    .number()
    .int("Los créditos deben ser un número entero")
    .min(1, "Mínimo 1 crédito")
    .max(10, "Máximo 10 créditos"),

  semestre: z
    .string()
    .min(1, "El semestre es obligatorio")
    .max(20, "El semestre no puede exceder 20 caracteres"),

  docente_id: z
    .string()
    .uuid("ID de docente inválido")
    .nullable()
    .optional(),
});

// Tipo inferido para usar en formularios y API
export type MateriaFormData = z.infer<typeof materiaSchema>;
```

2.2 — Explicación del esquema

Campo	Validación	Ejemplo válido
codigo	3-10 chars, solo A-Z, 0-9, guiones, auto-uppercase	MAT-101, INF-200
nombre	3-100 chars, trim automático	Cálculo I, Programación Web
creditos	Entero entre 1 y 10	4, 6, 8
semestre	1-20 chars, obligatorio	2026-1, I-2026
docente_id	UUID válido, nullable y opcional	null o UUID de un docente

☑ Transform: La función `.transform()` modifica el valor después de validar. Así el código siempre se guarda en mayúsculas y el nombre sin espacios extra.

ℹ Reutilización: Definimos el esquema en un archivo separado (no dentro de la API ni del componente) para poder importarlo desde ambos lados sin duplicar código.

Paso 3 — API Route: Listar Materias (GET)

Este endpoint devuelve todas las materias con los datos del docente asignado. Solo admins pueden acceder.

3.1 — Crear la estructura

Terminal

```
mkdir -p app/api/admin/materias
```

3.2 — Crear la API Route

Crea el archivo `app/api/admin/materias/route.ts`:

app/api/admin/materias/route.ts

```
import { createClient } from "@lib/supabase/server";
import { createClient as createServiceClient } from "@supabase/supabase-js";
import { NextResponse } from "next/server";
import { materiaSchema } from "@lib/validations/materia";

// ===== GET: Listar todas las materias =====
export async function GET() {
  try {
    // 1. Verificar autenticación y rol admin
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "admin") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    // 2. Obtener materias con datos del docente (JOIN implícito)
    const supabaseAdmin = createServiceClient(
      process.env.NEXT_PUBLIC_SUPABASE_URL!,
      process.env.SUPABASE_SERVICE_ROLE_KEY!
    );
```



```

const { data: materias, error } = await supabaseAdmin
  .from("materias")
  .select(`
    id, codigo, nombre, creditos, semestre, docente_id, created_at,
    docente:usuarios_perfil!docente_id(id, nombre, apellido)
  `)
  .order("codigo", { ascending: true });

if (error) {
  console.error("Error al listar materias:", error);
  return NextResponse.json({ error: "Error al obtener materias" }, { status: 500 });
}

return NextResponse.json({ materias });
} catch (err) {
  console.error("Error inesperado:", err);
  return NextResponse.json({ error: "Error interno del servidor" }, { status: 500 });
}
}

```

3.3 — Formato de respuesta

El endpoint devuelve un JSON con esta estructura:

Ejemplo de respuesta JSON

```

{
  "materias": [
    {
      "id": "uuid-1",
      "codigo": "MAT-101",
      "nombre": "Cálculo I",
      "creditos": 6,
      "semestre": "2026-1",
      "docente_id": "uuid-docente",
      "created_at": "2026-04-01T...",
      "docente": {
        "id": "uuid-docente",
        "nombre": "María",
        "apellido": "López"
      }
    }
  ]
}

```

i JOIN implícito: Supabase detecta automáticamente la relación FK entre `materias.docente_id` y `usuarios_perfil.id`. El alias `docente` renombra el resultado para que sea más legible.

Paso 4 — API Route: Crear Materia (POST)

Ahora agregaremos el handler POST en el mismo archivo route.ts para crear nuevas materias con validación Zod en el servidor.

4.1 — Agregar el handler POST

Abre app/api/admin/materias/route.ts y agrega el siguiente código después de la función GET:

```
app/api/admin/materias/route.ts – agregar después del GET
// ===== POST: Crear nueva materia =====
export async function POST(request: Request) {
  try {
    // 1. Verificar autenticación y rol admin
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "admin") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    // 2. Leer y validar el body con Zod
    const body = await request.json();
    const resultado = materiaSchema.safeParse(body);

    if (!resultado.success) {
      const errores = resultado.error.issues.map((issue) => ({
        campo: issue.path.join("."),
        mensaje: issue.message,
      }));
      return NextResponse.json(
        { error: "Datos inválidos", errores },
        { status: 400 }
      );
    }

    // 3. Insertar con Service Role
    const supabaseAdmin = createServiceClient(
      process.env.NEXT_PUBLIC_SUPABASE_URL!,
      process.env.SUPABASE_SERVICE_ROLE_KEY!
    );
```

```

const { data, error } = await supabaseAdmin
  .from("materias")
  .insert({
    codigo: resultado.data.codigo,
    nombre: resultado.data.nombre,
    creditos: resultado.data.creditos,
    semestre: resultado.data.semestre,
    docente_id: resultado.data.docente_id || null,
  })
  .select(`
    id, codigo, nombre, creditos, semestre, docente_id,
    docente:usuarios_perfil!docente_id(id, nombre, apellido)
  `)
  .single();

if (error) {
  console.error("Error al crear materia:", error);

  // Error de código duplicado
  if (error.code === "23505") {
    return NextResponse.json(
      { error: "Ya existe una materia con ese código" },
      { status: 409 }
    );
  }

  return NextResponse.json(
    { error: "Error al crear la materia" },
    { status: 500 }
  );
}

return NextResponse.json({ materia: data }, { status: 201 });
} catch (err) {
  console.error("Error inesperado:", err);
  return NextResponse.json(
    { error: "Error interno del servidor" },
    { status: 500 }
  );
}
}

```

4.2 — Puntos clave del POST

Concepto	Detalle
safeParse	A diferencia de parse(), no lanza excepción. Devuelve { success, data } o { success, error }
Errores Zod formateados	Mapeamos error.issues a un formato {campo, mensaje} legible para el frontend
Código 23505	Error de PostgreSQL para violación de UNIQUE constraint (código duplicado)

Concepto	Detalle
Status 201	Created — el código HTTP correcto para recursos nuevos
Status 409	Conflict — indica que el recurso ya existe
SELECT después de INSERT	.select() después de .insert() devuelve el registro recién creado con datos del JOIN

☒ **safeParse vs parse:** Usa safeParse() en API Routes para controlar los errores. Usa parse() en formularios con React Hook Form porque el zodResolver ya maneja las excepciones.

Paso 5 — API Route: Listar Docentes (GET)

Necesitamos un endpoint auxiliar que devuelva la lista de docentes activos. Se usará para llenar el Select de "Asignar docente" en el formulario de crear materia.

5.1 — Crear el archivo

Terminal

```
mkdir -p app/api/admin/docentes
```

Crea el archivo `app/api/admin/docentes/route.ts`:

app/api/admin/docentes/route.ts

```
import { createClient } from "@lib/supabase/server";
import { createClient as createServiceClient } from "@supabase/supabase-js";
import { NextResponse } from "next/server";

export async function GET() {
  try {
    // 1. Verificar autenticación y rol admin
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "admin") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    // 2. Obtener docentes activos
    const supabaseAdmin = createServiceClient(
      process.env.NEXT_PUBLIC_SUPABASE_URL!,
      process.env.SUPABASE_SERVICE_ROLE_KEY!
    );

    const { data: docentes, error } = await supabaseAdmin
      .from("usuarios_perfil")
      .select("id, nombre, apellido")
      .eq("rol", "docente")
      .eq("activo", true)
```

```
        .order("apellido", { ascending: true });

    if (error) {
        console.error("Error al listar docentes:", error);
        return NextResponse.json({ error: "Error al obtener docentes" }, { status: 500 });
    }

    return NextResponse.json({ docentes });
} catch (err) {
    console.error("Error inesperado:", err);
    return NextResponse.json({ error: "Error interno del servidor" }, { status: 500 });
}
}
```

i Filtro doble: Filtramos por rol = "docente" Y activo = true. No queremos mostrar docentes desactivados como opción para asignar a materias.

Paso 6 — Componente: Tabla de Materias con Formulario de Alta

Este componente es similar al UsuariosTable de la Entrega 4B, pero incluye además un formulario modal para crear materias. Lo construiremos en secciones.

6.1 — Crear la carpeta

Terminal

```
# La carpeta admin ya existe de la Entrega 4B
# Solo necesitamos crear el archivo
```

6.2 — Sección A: Imports y tipos

Crea el archivo components/dashboard/admin/materias-table.tsx:

components/dashboard/admin/materias-table.tsx — Parte 1/5

```
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";

// Componentes shadcn/ui
import {
  Table, TableBody, TableCell, TableHead,
  TableHeader, TableRow,
} from "@/components/ui/table";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from "@/components/ui/select";
import {
  Dialog, DialogContent, DialogDescription,
  DialogFooter, DialogHeader, DialogTitle,
  DialogTrigger,
} from "@/components/ui/dialog";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";

// Iconos
import { BookOpen, Plus, RefreshCw } from "lucide-react";

// Tipos
interface Docente {
  id: string;
  nombre: string;
  apellido: string;
}
```

```
interface Materia {  
  id: string;  
  codigo: string;  
  nombre: string;  
  creditos: number;  
  semestre: string;  
  docente_id: string | null;  
  created_at: string;  
  docente: Docente | null;  
}
```

i Tipo docente anidado: La interfaz Materia incluye un campo docente que es un objeto Docente | null, reflejando exactamente la estructura del JSON que devuelve nuestra API.

6.3 — Sección B: Estado y carga de datos

components/dashboard/admin/materias-table.tsx — Parte 2/5

```
export default function MateriasTable() {
  // Estados de datos
  const [materias, setMaterias] = useState<Materia[]>([]);
  const [docentes, setDocentes] = useState<Docente[]>([]);
  const [loading, setLoading] = useState(true);

  // Estados de filtro
  const [filtroSemestre, setFiltroSemestre] = useState<string>("todos");

  // Estados del formulario de crear
  const [dialogOpen, setDialogOpen] = useState(false);
  const [creando, setCreando] = useState(false);
  const [formData, setFormData] = useState({
    codigo: "",
    nombre: "",
    creditos: 4,
    semestre: "",
    docente_id: "",
  });

  // Cargar materias
  const cargarMaterias = async () => {
    setLoading(true);
    try {
      const res = await fetch("/api/admin/materias");
      const data = await res.json();
      if (!res.ok) { toast.error(data.error || "Error al cargar materias"); return; }
      setMaterias(data.materias);
    } catch {
      toast.error("Error de conexión");
    } finally {
      setLoading(false);
    }
  };

  // Cargar docentes (para el Select del formulario)
  const cargarDocentes = async () => {
    try {
      const res = await fetch("/api/admin/docentes");
      const data = await res.json();
      if (res.ok) setDocentes(data.docentes);
    } catch {
      console.error("Error al cargar docentes");
    }
  };

  useEffect(() => {
    cargarMaterias();
    cargarDocentes();
  }, []);

  // Obtener semestres únicos para el filtro
  const semestres = [...new Set(materias.map((m) => m.semestre))].sort();

  // Filtrar materias
  const materiasFiltradas = materias.filter((m) => {
    return filtroSemestre === "todos" || m.semestre === filtroSemestre;
  });
}
```

☑ **Set para valores únicos:** Usamos new Set() para extraer los semestres sin duplicados de las materias existentes. Esto genera el filtro de semestre dinámicamente.

6.4 — Sección C: Función para crear materia

```
components/dashboard/admin/materias-table.tsx — Parte 3/5
// Crear materia
const crearMateria = async () => {
  setCreando(true);
  try {
    const res = await fetch("/api/admin/materias", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        ...formData,
        creditos: Number(formData.creditos),
        docente_id: formData.docente_id || null,
      }),
    });

    const data = await res.json();

    if (!res.ok) {
      // Mostrar errores de validación
      if (data.errores) {
        data.errores.forEach(
          (e: { campo: string; mensaje: string }) =>
            toast.error(`${e.campo}: ${e.mensaje}`)
        );
      } else {
        toast.error(data.error || "Error al crear materia");
      }
      return;
    }

    toast.success(`Materia "${data.materia.codigo}" creada`);
    setDialogOpen(false);
    setFormData({
      codigo: "", nombre: "", creditos: 4,
      semestre: "", docente_id: "",
    });
    cargarMaterias();
  } catch {
    toast.error("Error de conexión");
  } finally {
    setCreando(false);
  }
};
```

Observa el manejo de errores de validación: si el servidor devuelve el array errores (de Zod), mostramos un toast por cada error. Si es un error general, mostramos un solo toast.

6.5 — Sección D: Renderizado — Filtros y botón crear

components/dashboard/admin/materias-table.tsx — Parte 4/5

```
return (
  <div className="space-y-4">
    {/* ===== BARRA SUPERIOR: FILTROS + BOTÓN CREAR ===== */}
    <div className="flex items-center gap-4">
      <Select value={filtroSemestre} onValueChange={setFiltroSemestre}>
        <SelectTrigger className="w-[200px]">
          <SelectValue placeholder="Filtrar por semestre" />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="todos">Todos los semestres</SelectItem>
          {semestres.map((s) => (
            <SelectItem key={s} value={s}>
              {s}
            </SelectItem>
          ))}
        </SelectContent>
      </Select>

      <Button variant="outline" size="sm" onClick={cargarMaterias}>
        <RefreshCw className="mr-2 h-4 w-4" />
        Refrescar
      </Button>

      <span className="text-sm text-muted-foreground">
        {materiasFiltradas.length} materia(s)
      </span>

      {/* Botón que abre el Dialog */}
      <div className="ml-auto">
        <Dialog open={dialogOpen} onOpenChange={setDialogOpen}>
          <DialogTrigger asChild>
            <Button>
              <Plus className="mr-2 h-4 w-4" />
              Nueva Materia
            </Button>
          </DialogTrigger>

          {/* ===== FORMULARIO MODAL ===== */}
          <DialogContent className="sm:max-w-[500px]">
            <DialogHeader>
              <DialogTitle>Crear Nueva Materia</DialogTitle>
              <DialogDescription>
                Completa los datos de la materia. El código debe ser único.
              </DialogDescription>
            </DialogHeader>

            <div className="grid gap-4 py-4">
              {/* Código */}
              <div className="grid gap-2">
                <Label htmlFor="codigo">Código</Label>
                <Input
                  id="codigo"
                  placeholder="MAT-101"
                  value={formData.codigo}
                  onChange={(e) =>
                    setFormData({ ...formData, codigo: e.target.value.toUpperCase() })
                  }
                />
              </div>
            </div>
          </DialogContent>
        </Dialog>
      </div>
    </div>
  )
```

```

{ /* Nombre */}
<div className="grid gap-2">
  <Label htmlFor="nombre">Nombre</Label>
  <Input
    id="nombre"
    placeholder="Cálculo Diferencial"
    value={formData.nombre}
    onChange={(e) =>
      setFormData({ ...formData, nombre: e.target.value })
    }
  />
</div>

{ /* Créditos y Semestre en fila */}
<div className="grid grid-cols-2 gap-4">
  <div className="grid gap-2">
    <Label htmlFor="creditos">Créditos</Label>
    <Input
      id="creditos"
      type="number"
      min={1}
      max={10}
      value={formData.creditos}
      onChange={(e) =>
        setFormData({
          ...formData,
          credits: parseInt(e.target.value) || 1,
        })
      }
    />
  </div>
  <div className="grid gap-2">
    <Label htmlFor="semestre">Semestre</Label>
    <Input
      id="semestre"
      placeholder="2026-1"
      value={formData.semestre}
      onChange={(e) =>
        setFormData({ ...formData, semestre: e.target.value })
      }
    />
  </div>
</div>

{ /* Docente (Select dinámico) */}
<div className="grid gap-2">
  <Label>Docente (opcional)</Label>
  <Select
    value={formData.docente_id || "sin-asignar"}
    onChange={(v) =>
      setFormData({ ...formData, docente_id: v === "sin-asignar" ? "" : v })
    }
  >
    <SelectTrigger>
      <SelectValue placeholder="Seleccionar docente" />
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="sin-asignar">Sin asignar</SelectItem>
      {docentes.map((d) => (
        <SelectItem key={d.id} value={d.id}>
          {d.nombre} {d.apellido}
        </SelectItem>
      ))}
    </SelectContent>
  </div>

```

```

        )))
      </SelectContent>
    </Select>
  </div>
</div>

<DialogFooter>
  <Button
    variant="outline"
    onClick={() => setDialogOpen(false)}
  >
    Cancelar
  </Button>
  <Button onClick={crearMateria} disabled={creando}>
    {creando ? "Creando..." : "Crear Materia"}
  </Button>
</DialogFooter>
</DialogContent>
</Dialog>
</div>
</div>

```

⚠ Select vacío: shadcn/ui Select no acepta `value=""` como valor válido. Por eso usamos "sin-assignar" como valor por defecto y lo convertimos a cadena vacía al guardar en `formData`.

6.6 — Sección E: Renderizado — Tabla

components/dashboard/admin/materias-table.tsx — Parte 5/5

```

{ /* ===== TABLA ===== */}
<div className="rounded-md border">
  <Table>
    <TableHeader>
      <TableRow>
        <TableHead>Código</TableHead>
        <TableHead>Nombre</TableHead>
        <TableHead>Créditos</TableHead>
        <TableHead>Semestre</TableHead>
        <TableHead>Docente</TableHead>
      </TableRow>
    </TableHeader>
    <TableBody>
      {loading ? (
        <TableRow>
          <TableCell colSpan={5} className="text-center py-8">
            Cargando materias...
          </TableCell>
        </TableRow>
      ) : materiasFiltradas.length === 0 ? (
        <TableRow>
          <TableCell colSpan={5} className="text-center py-8">
            No se encontraron materias
          </TableCell>
        </TableRow>
      ) : (
        materiasFiltradas.map((m) => (
          <TableRow key={m.id}>
            <TableCell className="font-mono font-medium">
              {m.codigo}
            </TableCell>
            <TableCell>{m.nombre}</TableCell>
            <TableCell>
              <Badge variant="secondary">{m.creditos}</Badge>
            </TableCell>
            <TableCell>
              <Badge variant="outline">{m.semestre}</Badge>
            </TableCell>
            <TableCell>
              {m.docente ? (
                `${m.docente.nombre} ${m.docente.apellido}`
              ) : (
                <span className="text-muted-foreground italic">
                  Sin asignar
                </span>
              )}
            </TableCell>
          </TableRow>
        ))
      )}
    </TableBody>
  </Table>
</div>
</div>
);
}

```

El código de la materia se muestra con font-mono para diferenciarlo visualmente como un código técnico. Los créditos y semestre usan Badge con variantes distintas para crear una jerarquía visual clara.

Paso 7 — Página Admin: Materias

Reemplazaremos la página placeholder de admin/materias con la página real.

7.1 — Actualizar la página

Reemplaza completamente el contenido de `app/(dashboard)/admin/materias/page.tsx`:

```
app/(dashboard)/admin/materias/page.tsx
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@components/ui/card";
import { BookOpen } from "lucide-react";
import MateriasTable from "@components/dashboard/admin/materias-table";

export default function AdminMateriasPage() {
  return (
    <div className="flex flex-1 flex-col gap-4 p-4">
      { /* Encabezado */ }
      <div className="flex items-center gap-3">
        <BookOpen className="h-8 w-8 text-primary" />
        <div>
          <h1 className="text-2xl font-bold tracking-tight">
            Gestión de Materias
          </h1>
          <p className="text-muted-foreground">
            Crea, visualiza y administra las materias del sistema académico.
          </p>
        </div>
      </div>

      { /* Card con la tabla */ }
      <Card>
        <CardHeader>
          <CardTitle>Catálogo de Materias</CardTitle>
          <CardDescription>
            Listado de todas las materias registradas con su docente asignado
          </CardDescription>
        </CardHeader>
        <CardContent>
          <MateriasTable />
        </CardContent>
      </Card>
    </div>
  );
}
```

☒ **Mismo patrón:** Observa cómo esta página sigue exactamente la misma estructura que `AdminUsuariosPage` de la Entrega 4B: Server Component con Card + componente interactivo dentro.

Resumen de Archivos

En esta entrega creaste o modificaste los siguientes archivos:

Archivo	Acción	Descripción
components/ui/dialog.tsx	Nuevo (shadcn)	Componente Dialog auto-generado
components/ui/textarea.tsx	Nuevo (shadcn)	Componente Textarea auto-generado
lib/validations/materia.ts	Nuevo	Esquema Zod + tipo MateriaFormData
app/api/admin/materias/route.ts	Nuevo	GET: listar materias + POST: crear materia
app/api/admin/docentes/route.ts	Nuevo	GET: listar docentes activos
components/dashboard/admin/materias-table.tsx	Nuevo	Tabla + filtros + formulario modal
app/(dashboard)/admin/materias/page.tsx	Modificado	Reemplaza placeholder con MateriasTable

Verificación Final

Ejecuta npm run dev y verifica cada uno de estos puntos:

A. Acceso

- ✓ Iniciar sesión como admin y navegar a /admin/materias
- ✓ La página muestra el encabezado y la Card con la tabla

B. Tabla vacía

- ✓ Si no hay materias, se muestra "No se encontraron materias"
- ✓ El filtro de semestre está en "Todos los semestres"

C. Crear materia

- ✓ Clic en "Nueva Materia" abre el diálogo modal
- ✓ El formulario tiene campos: código, nombre, créditos, semestre, docente
- ✓ El Select de docente muestra la lista de docentes activos
- ✓ Crear una materia con datos válidos muestra toast de éxito
- ✓ La tabla se actualiza automáticamente con la nueva materia
- ✓ El diálogo se cierra y el formulario se limpia

D. Validaciones

- ✓ Código con menos de 3 caracteres muestra error
- ✓ Código duplicado muestra "Ya existe una materia con ese código"
- ✓ Créditos fuera del rango 1-10 muestra error
- ✓ Semestre vacío muestra error

E. Filtros

- ✓ Después de crear varias materias con semestres diferentes, el filtro los muestra
- ✓ Filtrar por semestre específico muestra solo las materias de ese semestre
- ✓ El contador se actualiza con los resultados filtrados

F. Datos del docente

- ✓ Materias con docente muestran el nombre completo en la tabla
- ✓ Materias sin docente muestran "Sin asignar" en itálica

Ejercicio Propuesto

Practica lo aprendido con estas tareas adicionales:

1. Búsqueda por texto: Agrega un Input que filtre materias por código o nombre. Combina este filtro con el filtro de semestre existente. Usa el método `.toLowerCase().includes()` para hacer la búsqueda insensible a mayúsculas.
2. Validación en el cliente: Agrega React Hook Form + zodResolver al formulario del Dialog, igual que hiciste con el perfil en la Entrega 4A. Muestra los errores inline debajo de cada campo usando `FormMessage`.
3. Estadísticas con `StatsCard`: Arriba de la tabla, agrega 3 `StatsCard` mostrando: Total de materias, Total de créditos ofertados (suma de todos los créditos) y Materias sin docente asignado.
4. Datos de prueba: Crea al menos 8 materias de ejemplo con datos realistas (usa códigos como MAT-101, INF-201, FIS-301, etc.) distribuidas en 2 semestres diferentes. Asigna docentes a al menos 5 de ellas. Toma capturas de pantalla de la tabla con filtros aplicados.

i Para entregar: Implementa al menos 2 de los 4 ejercicios y crea los datos de prueba del ejercicio 4. Muestra capturas de pantalla en clase.

Próximo: Entrega 5B

En la siguiente entrega completaremos el CRUD de materias:

- API Route PUT /api/admin/materias/[id] para editar una materia existente
- API Route DELETE /api/admin/materias/[id] para eliminar una materia
- Formulario de edición que precarga los datos actuales de la materia
- Botón de eliminar con AlertDialog de confirmación
- Columna de acciones en la tabla con botones Editar y Eliminar

☒ **Preparación:** Asegúrate de tener al menos 3 materias creadas con el formulario de esta entrega. Las necesitarás para probar la edición y eliminación en la Entrega 5B.

Fin de la Entrega 5A